

Retrieval Augmented Generation

Elias W.J. Müller

FS26

Contents

1	RAG Introduction	2
2	Information Retrieval	2
2.1	Sparse Encoding	2
2.1.1	Binary Indexing	2
2.1.2	Term-Frequency Inverse Document-Frequency (TF-IDF)	2
2.1.3	BM25	3
2.1.4	Inverted Index	3
2.1.5	BM25 ranking with Inverted Index	4
2.2	Dense Retrieval	4
2.2.1	Dense Encoding: Vector Space Model	4
2.2.2	Dense Encoding: Nearest Neighbor Search	4
2.2.3	Dense Encoding: Locality-Sensitive Hashing	5
2.2.4	Dense Retrieval Learned Embeddings: Optimization Objectives	6
2.2.5	Dense Retrieval Learned Embeddings: Document Sampling Strategies	7
2.3	Information Retrieval Evaluation	8
3	Language Modeling	9
3.1	LLM Basics	9
3.1.1	Count-Based LM	9
3.1.2	Auto-regressive LM	10
3.1.3	LM Performance Evaluation	11
3.2	LM Architectures	11
3.3	Scaling Law	12
3.4	LLM Paradigms	12
3.4.1	Learning Paradigms	12
3.4.2	Reasoning Paradigms	15
4	RAG Engineering	17
4.1	Backend Engineering	17
4.1.1	Database Management	17
4.1.2	System Design & APIs	18
4.1.3	Deployment	19
4.2	Frontend Engineering	19
4.2.1	Design Principles	19
4.2.2	React.JS	20
4.2.3	Human-Computer Interaction	20
4.3	System Testing	21
4.3.1	Development Testing	21
4.3.2	Release Testing	22
4.3.3	User Testing	22
4.3.4	Software Evolution	22
4.4	RAG Frameworks & Applications	23
4.4.1	Frameworks	23
4.4.2	Application Patterns	23

1 RAG Introduction

RAG = Retrieval (fetch relevant context) + Generation (condition an LLM on it), to combat hallucinations. This summary follows that split: **Information Retrieval** (the “R”), **Language Modeling** (the “G”), then **RAG Engineering** (building, serving, testing, and extending the system).

Motivation

- **Extrinsic LLM Hallucinations:** non factual to training data (wrong recall)
- **Intrinsic LLM Hallucinations:** non faithful to the input context, i.e., false retrieval from context (even if factual)

Many methods exist at different stages:

- RAG (Inference): partial intrinsic hallucinations but primarily for extrinsic. But retrieval failures propagate or the model might ignore retrieved context.
- Instruction Tuning/ RLHF (Training): only for extrinsic hallucinations, but encourages fluency over faithfulness, and has no approach against source errors
- Contrastive Sampling (Training): Both intrinsic and extrinsic, but requires expensive hallucination labeled data
- HITL (Post-Generation): Both intrinsic and extrinsic, but not scalable and expensive

2 Information Retrieval

The “R” in RAG: find the documents relevant to a query. Two paradigms (*sparse* then *dense*), then how to evaluate a retriever. The distinction Sparse vs. Dense does not relate to data storage, but rather to the conceptual approach. *Sparse* methods are the non-neural, statistical approaches (we count) whereas *Dense* approaches rely on neural encoding methods incorporation semantic information.

2.1 Sparse Encoding

Non-neural, statistical approaches that generally result in a sparse term-document representation in the query logic. To be used whenever speed and precision (in the metric sense) matters. But sparse encodings have a recall issue.

2.1.1 Binary Indexing

Trivial binary indication as *vectorized* representations of queries in *Term-Document* matrix $\in \mathbb{R}^{m \times n}$ (each term $\times$ each document section (e.g. chapter)), with m terms (*vocabulary* size) and n documents (*corpus* size). Indexing by creating a data structure that functions as a look-up-table for representing the terms we want to search over. Generally, a RegEx approach that is *fast and efficient* and *re-executable* BUT:

- **Very limited flexibility** in terms of expressiveness - only boolean queries (e.g. AND, OR, NOT): leverage vectors term for binary ops: $(1, 0, 0, 1)$ AND $(1, 1, 0, 0) \Rightarrow (1, 0, 0, 0)$
- **High memory consumption** with extremely sparse representation in term-document matrix/ DB table $(m \times n)/8 \text{ bits} = \text{bytes}$
- **Limited representativeness:** no relevance ranking since all terms are equally important

2.1.2 Term-Frequency Inverse Document-Frequency (TF-IDF)

A Bag-of-Words (BoW) approach representing the term-document matrix with number of appearances in the corpus. Based on the probabilistic idea that term appearances make documents prototypical.

High TF-IDF weights are thus assigned to a term t if it appears only in a few documents in the corpus but in these with high frequency (discriminative), and low weights if t appears in most/all documents and is thus not discriminative.

TF-IDF combines TF and IDF we get: $\text{TF-IDF}_{t,d} = \text{TF}_{t,d} \times \text{IDF}_t$

- **Term Frequency** term counts normalized by the number of terms in the documents after *stop-word* removal:

$$\text{tf}_{t,d} = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

Isomorphic approach, i.e. BoW ignores word order of the term appearances. Essentially, we get high weights for words that appear frequently.

- **Inverse Document Frequency** assigns different weights to different words, giving *low weight to common less indicative words* and *high weights to rare and more indicative words* in terms of the corpus at hand.

$$\text{Document Frequency (by term weight for document appearance):} \quad \text{df}_t = \sum_{d \in \mathbb{D}} 1, \text{ if } t \in d$$

$$\text{Inverse Document Frequency:} \quad \text{idf}_t = \log \frac{\sum_{d \in \mathbb{D}} 1}{\text{df}_t}, \text{ if } t \in d$$

We use the IDF for computational efficiency, since df-values can be very large for large corpora.

Since the vocabulary V is compiled from the corpus $\mathbb{D}$ we can only represent tokens in V . With n documents

$$1 \leq \text{df}_t \leq n \implies 1 \leq \frac{n}{\text{df}_t} \leq n \implies 0 \leq \text{idf}_t \leq \log n$$

However, idf can only be 0 for Query-Terms that are Out-Of-Domain and not for tokens in V . The *base of the logarithm* does not affect document ranking — it rescales all idf-values by a constant and thus preserves their order.

Drawbacks of TF-IDF:

- **Linear Scaling:** TF grows linearly with raw count, but importance has diminishing returns — going from 1→2 occurrences is more meaningful than 99→100. TF-IDF has no saturation mechanism and thus over-rewards high-frequency terms
- **Varying document lengths:** longer documents have higher TF due to more words, not higher relevance
- **Rigid:** static formula with no adjustable parameters → cannot adapt to specific corpora or tasks
- **Count-based:** does not utilize prior knowledge about the corpus — essentially fancy RegEx

2.1.3 BM25

BM25 (Best Matching 25) augments TF-IDF with advanced frequency terms, document length normalization and prevention of extremes, to rank documents by their probability of relevance (*Probability Ranking Principle*). It's the SOTA Benchmark as a tunable and controllable approach:

$$\text{BM25}_{q,d} = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f_{q_i,d} \cdot (k_1 + 1)}{f_{q_i,d} + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avg. dl}}\right)}$$

where $f_{q_i,d}$ is the term frequency of q_i in d , $|d|$ the document length, $k_1 \in [1.2, 2.0]$ a TF saturation parameter, $b = 0.75$ a length normalization parameter, and

$$\text{IDF}(q_i) = \log \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$

with N the corpus size, $n(q_i)$ the number of documents containing q_i , and 0.5 a smoothing term to avoid $\text{df}_t = 0$ for query terms absent from the corpus. This also makes it less sensitive to term frequencies, thus resulting in a less pronounced linear scaling effect (more saturation).

Memory and Storage Challenges are, however, a common issue with BM25 thus we required an *Inverted Index* for large scale corpora settings.

2.1.4 Inverted Index

The mapping $f : d \mapsto t$ is reversed, s.t., $g : t \mapsto d$. Wrt. the TD-Matrix this means that we no longer consider the Term-Document mapping, but store **Term** $\mapsto$ **Inverted Linked List** (Chain of Document IDs — TFs) → No more zeros and space-efficient storage via data structs like `json`

Algorithm 1 BM25 Ranking: Ranking and Returning Results

Input: Documents $\mathcal{D}$, query $\mathcal{Q} = \{q_1, \dots, q_n\}$, params k_1, b , cutoff k

Output: Ranked list of the top k documents

Initialize: empty inverted index I , empty list of doc lengths L , doc count N

Step 1: Build Inverted Index

foreach document D_i **do**

 Split D_i into terms $\{t_1, t_2, \dots\}$
 Compute term frequencies for D_i
 Update $I[t]$ with (D_i, tf) for each term t
 Compute length $L[D_i] = \text{len}(D_i)$

Compute average document length $\text{avg.dl} = \frac{\sum L[D_i]}{N}$; // normalize

Step 2: Compute BM25 Scores;

// init empty dictionary scores

foreach term $q \in \mathcal{Q}$ **do**

if $q \in I$ **then**
 $\text{idf}(q) \leftarrow \log\left(\frac{N - |I[q]| + 0.5}{|I[q]| + 0.5}\right)$
 foreach $(D_i, \text{tf}) \in I[q]$ **do**
 $L_d \leftarrow L[D_i]$
 $\text{score} \leftarrow \text{idf}(q) \cdot \frac{\text{tf} \cdot (k_1 + 1)}{\text{tf} + k_1 \cdot \left(1 - b + b \cdot \frac{L_d}{\text{avg.dl}}\right)}$
 $\text{scores}[D_i] \leftarrow \text{scores}[D_i] + \text{score}$

Step 3: Rank Documents

Sort *scores* in descending order

return top k documents with the highest scores

2.1.5 BM25 ranking with Inverted Index

Given $\mathcal{D}$ documents and query $\mathcal{Q} = \{q_1, \dots, q_n\}$ we return a ranked list of top k documents.

2.2 Dense Retrieval

Neural encoding into a shared vector space to capture the *semantics* sparse methods miss: the vector space model, (approximate) nearest-neighbor search to make it scale, and how the embeddings are learned.

2.2.1 Dense Encoding: Vector Space Model

Motivation: While sparse encodings are easy and interpretable, they do not model the semantics.

Trivial approach: use the TF-vectorized document representations (documents x term (count) matrix) and use some similarity metric:

- Vector Difference $e_{d_1} - e_{d_2}$: very easy but sensitive to absolute differences in TF and not normalized wrt. doc length (longer docs have more diversity naturally)
- Cosine Similarity

$$\text{sim}(e_{x_1}, e_{x_2}) = \frac{e_{x_1} \cdot e_{x_2}}{\|e_{x_1}\| \|e_{x_2}\|}$$

we could apply this to some e_{d_1}, e_{d_2} , but we want to find relevant documents with a **query**: $\text{sim}(e_q, e_d)$. The inner product, however, cannot be computed given $e_q \in \mathbb{R}^{1 \times m}$ (TF-IDF row) and $e_d \in \mathbb{R}^{n \times 1}$ (TF-IDF col)

→ *embed query and document in the same dimensionality* using Word Embeddings (Word2Vec) and representing e_q, e_d as the average of their word embeddings. (doc embeddings can be precomputed and stored in RAM for efficiency)

2.2.2 Dense Encoding: Nearest Neighbor Search

How to apply the Cosine Similarity.

Nearest Neighbor Search *Brute-Force/ exhaustive* computation of all similarities: $\arg \max_{d_i \in D} \text{sim}(e_q, e_{d_i})$ in $\mathcal{O}(n)$ for one query $\mathcal{O}(m \cdot n)$ for m queries. Operations with complexity $n \cdot k$ since inner product.

$$\text{Matrix-based Comp/ Score Matrix: } S = \frac{M_Q M_D}{\|M_Q\|_2 \|M_D\|_2} \quad S_{i,j} \in [-1, 1],$$

Approximate Nearest Neighbor Search Given set of documents D , a text encoder e s.t. each output $\mathbb{R}^k$, find documents $D^* \subseteq D$ s.t. (with tolerance ϵ):

$$\forall d_i^* \in D^* \implies \text{dist}(e_q, e_{d_i^*}) \leq (1 + \epsilon) \cdot \min_{d \in D} \text{dist}(e_q, e_d)$$

$\text{dist}(\cdot)$ can be any monotonic function:

- **Euclidean Distance (L_2):** $\text{dist}(e_q, e_d) = \sqrt{\sum_{i=1}^k (e_{q_i} - e_{d_i})^2}$ - straight-line distance; sensitive to magnitude
- **Cosine Distance:** $\text{dist}(e_q, e_d) = 1 - \frac{e_q \cdot e_d}{\|e_q\|_2 \|e_d\|_2}$ - angle-based, magnitude-invariant; standard for embeddings
- **Manhattan Distance (L_1):** $\text{dist}(e_q, e_d) = \sum_{i=1}^k |e_{q_i} - e_{d_i}|$ - more robust to outliers than L_2
- **Jaccard Distance:** $\text{dist}(b_q, b_d) = 1 - \frac{|b_q \cap b_d|}{|b_q \cup b_d|}$ - set overlap (intersection over union); for binary/sparse

Options for Approximate NNS

- *Model Quantization* to reduce space complexity with some performance degradation. (e.g. `fp32` $\rightarrow$ `int8`)
- *Matrix Factorization*: SVD or Canonical Correlation Analysis
- *Clustering-Based Approaches*: k -means or hierarchical clustering
- *Hashing-Based Approaches*: Locality-Sensitive Hashing (LSH)

2.2.3 Dense Encoding: Locality-Sensitive Hashing

We use Local Sensitivity (Bucketing) for approximate NNS. The key property is that hash functions are *distance-preserving*, i.e. $\text{dist}(x, y) \propto \text{dist}(h(x), h(y))$, so similar items collide into the same bucket. Given two doc embeddings, some similarity measure, a threshold s and a hash function h , then we can hash similar docs into the same bucket and only search within this:

- $\text{sim}(e_{d_1}, e_{d_2}) \geq s \rightarrow h(e_{d_1}) = h(e_{d_2})$
- $\text{sim}(e_{d_1}, e_{d_2}) < s \rightarrow h(e_{d_1}) \neq h(e_{d_2})$

Sampling Strategy

1. **Sampling Strategy:** Choose set of hash functions (e.g. Bit Sampling, i.e. given binary repr, a bit sampling h selects n indices: $h_1 = [1, 3, 4] \rightarrow h_1((1, 0, 0, 1, 1, 0)) = 011$).

For n -bit sampling you can have 2^n hash functions.

2. **Create Hash Table:** Apply the selected hashing functions to all documents and create the buckets (hash-values $\times$ hash-functions)
3. **NN search within:** Apply hash function to the vectorized repr of the query (e.g. binary if bit-sampling), and perform NN-search in each bucket that was matched by the hash function on the query:

$$\text{return } \arg \max_{d_i \in [d_3, d_6]} \text{sim}(e_q, e_{d_i}) \quad \text{for each matched bucket per hash function}$$

4. *Hyperparam selection* (K hash functions, L buckets):

- **Larger K (stricter hashing):** smaller buckets $\rightarrow$ fewer NN candidates, but more false negatives $\rightarrow$ lower Recall. Pick K so bucket size stays manageable.
- **Larger L (more redundancy):** more hash tables required $\rightarrow$ higher chance a true neighbor collides somewhere $\rightarrow$ fewer false negatives, higher Recall. Cost: repeated NN computation across tables + more memory to store the extra hash tables.

Hash function choice Bit sampling ignores the actual similarities when bucketing. Alternatively, use *random-projection (cosine) hashing*: given a random vector $e_{r_j} \in \mathbb{R}^k$ and threshold s ,

$$f(e_{r_j}, e_q) = \begin{cases} 1 & \text{if } \text{sim}(e_{r_j}, e_q) \geq s \\ 0 & \text{otherwise} \end{cases}, \quad h_i(e_q) = f(e_{r_j}, e_q), \quad \forall i \in [1, \dots, K], \forall j \in [1, l]$$

with the hash value formed over l such projections. Now the buckets do reflect similarity. Training the e_{r_j} instead of sampling them randomly yields *Semantic Hashing* (Salakhutdinov & Hinton 2007).

2.2.4 Dense Retrieval Learned Embeddings: Optimization Objectives

Encoder choice (pooling). A document embedding aggregates its word embeddings, either by *Mean Pooling* $e_d = \frac{1}{|d|} \sum_{w \in d} e_w$ or *Max Pooling* $e_{d_i} = \max_{w \in d} e_{w_i} \forall i \in [1, \dots, k]$. Off-the-shelf encoders (Sent2Vec, Sentence-BERT) optimize generic semantic similarity rather than IR and tend to inflate similarity scores $\rightarrow$ motivates IR-specific training.

The choice of embeddings matters a lot in training a model. While cosine similarity is common, recent research has uncovered limitations.

(1) Cosine similarity is not automatically meaningful (Steck et al. 2024).

- Similarity scores are an *artifact of regularization*, not an intrinsic property of the embeddings:
 - **Drop-out style:** penalizes the *product* of embedding matrices *jointly* $\rightarrow$ arbitrary / meaningless cosine scores.
 - **Weight-decay style:** penalizes each matrix *separately* $\rightarrow$ well-defined cosine scores.
- Modern models mix both (both are L_2 reg.) $\Rightarrow$ explains why cosine *sometimes* works, sometimes not.
- *Intuition:* reg. can *anisotropically* stretch/squeeze the space, so cosine ends up measuring *geometric distortion*, not semantics $\Rightarrow$ bad retriever $\Rightarrow$ “RAG fails because the retriever is bad”.
- *Takeaway:* the **training objective**, not the metric, decides whether geometry is semantically meaningful.

(2) What we want $\approx$ a margin objective (SVM analogy).

- Pull query *close* to relevant docs, push *far* from irrelevant ones.
- Same shape as soft-margin SVM: $\min_{\mathbf{w}, b} \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \max\{0, 1 - y_i(\mathbf{w}_i^T \mathbf{x}_i + b)\}$, $y_i \in \{-1, 1\}$.
- *Mapping:* separating hyperplane $\leftrightarrow$ relevant/irrelevant boundary; margin $\leftrightarrow$ how confidently pos beats neg; support vectors $\leftrightarrow$ hard (near-boundary) negatives.

(3) Concrete training objectives (contrastive losses). Setup: query $\mathbf{q}$, positive doc $\mathbf{d}^+$, negative doc $\mathbf{d}^-$.

- **Margin / triplet hinge** – *pairwise*:

$$\mathcal{L}_\theta = \sum_{\mathbf{d}^+ \in D^+} \sum_{\mathbf{d}^- \in D^-} \max(0, \text{sim}(\mathbf{q}, \mathbf{d}^+) - \text{sim}(\mathbf{q}, \mathbf{d}^-) + \epsilon)$$

ϵ = margin (required gap; controls sensitivity). *Sign note:* as written sim acts as a *distance*; for a true similarity (higher = better) flip to $\max(0, \epsilon - \text{sim}(\mathbf{q}, \mathbf{d}^+) + \text{sim}(\mathbf{q}, \mathbf{d}^-))$.

- **InfoNCE / softmax NLL** – 1 pos vs. n negs:

$$\mathcal{L}_\theta = -\log \frac{e^{\text{sim}(\mathbf{q}_i, \mathbf{p}_i^+)}}{e^{\text{sim}(\mathbf{q}_i, \mathbf{p}_i^+)} + \sum_{j=1}^n e^{\text{sim}(\mathbf{q}_i, \mathbf{p}_j^-)}}$$

Intuition: cross-entropy that wants the positive to “win” the softmax over all negatives. More negatives $\rightarrow$ harder, better-calibrated gradient.

- **Temperature-scaled InfoNCE:**

$$\mathcal{L}_\theta = -\log \frac{e^{\text{sim}(\mathbf{q}_i, \mathbf{p}_i^+)/\tau}}{e^{\text{sim}(\mathbf{q}_i, \mathbf{p}_i^+)/\tau} + \sum_{j=1}^n e^{\text{sim}(\mathbf{q}_i, \mathbf{p}_j^-)/\tau}}$$

τ = temperature. *Intuition:* small $\tau \rightarrow$ sharp softmax, focuses gradient on the *hardest* negatives; large $\tau \rightarrow$ softer, treats negatives more uniformly.

(4) **Qwen3-Embedding: a concrete modern recipe.** Loss = “souped-up” InfoNCE. Same InfoNCE skeleton as before, but the denominator Z_i pools *five* sources of negatives, each gated by a mask m :

$$\mathcal{L}_{\text{embedding}} = -\frac{1}{N} \sum_i \log \frac{e^{s(q_i, d_i^+)/\tau}}{Z_i}$$

$$Z_i = \underbrace{e^{s(q_i, d_i^+)/\tau}}_{(1) \text{ positive}} + \underbrace{\sum_k m_{ik} e^{s(q_i, d_{i,k}^-)/\tau}}_{(2) K \text{ hard negs}} + \underbrace{\sum_{j \neq i} m_{ij} e^{s(q_i, q_j)/\tau}}_{(3) \text{ in-batch } q-q} + \underbrace{\sum_{j \neq i} m_{ij} e^{s(d_i^+, d_j)/\tau}}_{(4) \text{ in-batch } d-d} + \underbrace{\sum_{j \neq i} m_{ij} e^{s(q_i, d_j)/\tau}}_{(5) \text{ in-batch } q-d}$$

- **Hard negatives** $d_{i,k}^-$: mined to be lexically similar (e.g. high BM25) but semantically wrong $\Rightarrow$ forces fine-grained discrimination, not just easy random rejects.
- **In-batch negatives** (3)–(5): reuse every other example in the batch “for free” as negatives across all pairings ($q-q$, $d-d$, $q-d$) $\Rightarrow$ huge effective negative count at no extra cost.
- **Mask factor** m = **false-negative guard** (the key “improvement”):

$$m_{ij} = \begin{cases} 0, & \text{if } s_{ij} > s(q_i, d_i^+) + 0.1 \text{ or } d_j = d_i^+ \\ 1, & \text{otherwise} \end{cases}$$

Intuition: an in-batch “negative” that scores *higher* than the true positive (by > 0.1) is almost certainly a duplicate / mislabeled positive $\Rightarrow$ zero it out so the loss never penalizes a correct match. Prevents positive leakage from batch construction.

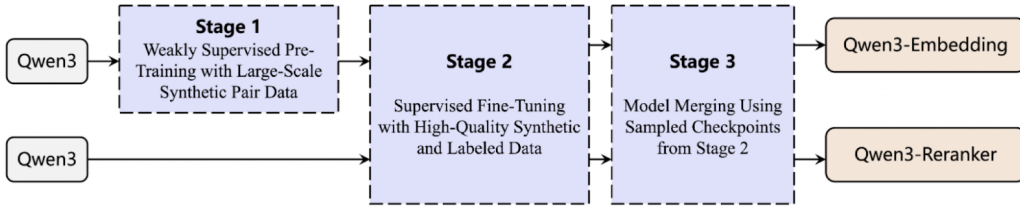


Figure 1: Qwen3-Embedding

2.2.5 Dense Retrieval Learned Embeddings: Document Sampling Strategies

Given query $\mathbf{q}$ and positive $\mathbf{d}^+$, how to pick negatives from D ? Five strategies on a *cheap/weak* $\rightarrow$ *expensive/informative* spectrum:

1. **Random:** $\mathbf{d}^- \sim D \setminus \{\mathbf{d}^+\}$ — cheap, but mostly trivial negatives + false-negative risk.
2. **In-Batch:** other queries’ positives in batch $\mathcal{B}$ as negatives — free, but quality depends on batch composition.
3. **Cross-Batch:** negatives from outside $\mathcal{B}$ (memory queue) — more diverse, but needs extra memory.
4. **Hard Negative:** top_k by $\text{sim}(\mathbf{q}, \mathbf{d}) > \tau$ — similar-but-wrong docs give strong gradient, but expensive + overfitting + *highest* false-negative risk.
5. **Semantic-Aware:** negatives sharing prior-knowledge features $\phi(\mathbf{d}) = \phi(\mathbf{d}^+)$ — context-relevant, but needs metadata ϕ .

Key threads: (i) *false negatives* plague 1, 2, 4 (worst for hard negatives, since top-similar docs are often unlabeled positives) — this is exactly what Qwen3’s mask m_{ij} fixes. (ii) Strategy 4 = *dynamic* hardness (model-scored, drifts, re-mine), Strategy 5 = *static* hardness (fixed features). (iii) τ here is a *similarity threshold*, not the softmax temperature.

2.3 Information Retrieval Evaluation

Expectation: a good retriever must have *high recall and precision, fast response, and robust query understanding* (handle varying complexity, formulations and ambiguities)

- **Precision:** $tp/(tp + fp)$ (Precision@k) - ignores order of retrieval
- **Recall:** $tp/(tp + fn)$ (Recall@k) - ignores order of retrieval
- **Accuracy:** $(tp + tn)/n$ - not used, since we often don't care about irrelevance identification performance, and imbalanced datasets (most irrelevant)
- **Mean Reciprocal Rank:** order/ rank of relevant retrieval (0.5 if at second position since $1/2$). The mean simply averages over a set of queries in an IR test set

$$MRR = \frac{1}{n} \sum_{i=1}^n RR(q_i) \quad \text{with} \quad RR(q) = \frac{1}{\text{rank of the first relevant retrieval}}$$

While MRR is *indicative* (the higher the better) and *interpretable*, it is biased since only *the first relevant matters*, and it is *narrow* (only suitable if only one relevant document per query). Importantly, it has no explanation of the relative *ranking quality* (if multiple relevant docs exist) - the most relevant can be very low

- **Normalized Discounted Cumulative Gain NDCG@k:** compares ranking of *relevance scores* to ideal GR ranking with cut-off k :

$$NDCG@k = \frac{DCG@k}{IDCG@k} \quad \text{where} \quad DCG@k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)}$$

rel_i being some relevance score (cosine sim. or binary) and DCG@k: discounted CG with IR ranking and IDCG@k: ideal discounted CG with GT ranking. Note $CG@k = \sum_{i=1}^k rel_i$ but we use DCG since logarithmic discount assigns lower gain when relevant items appear further down (difference 1 to 2 is larger than difference 2 to 3).

Pros: rank aware, variable relevance scores, and normalized

Cons: relevance score selection dependent, requires GT (not stand-alone eval metric)

- **Miscellaneous** (online/behavioral metrics, log-based, no GT relevance labels needed):
 - **Click-Through Rate (CTR):** Clicks/Impressions - ratio of users who click vs. those who only see it
 - **Session Success Rate (SSR):** Successful Sessions/Total Sessions - ratio of sessions where the user achieves their goal (e.g. click, find answer)
 - **Zero Result Rate (ZRR):** Zero Result Queries/Total Queries - ratio of queries returning no results

3 Language Modeling

The “G” in RAG: model the probability of text to generate answers. The arc: from count-based n-grams to neural autoregressive LMs (*basics*), the *architectures* that realize them, their *scaling* behavior, and the learning/reasoning *paradigms* built on top.

3.1 LLM Basics

An LM assigns probabilities to token sequences such that proper language scores higher, e.g. $P(\text{the mouse ate the cheese}) \gg P(\text{the the ate cheese mouse})$. Model these probabilities and we can generate. Probabilistic natural language modeling form a sequence of n tokens $X = (x_1, \dots, x_n)$ given a vocabulary V , basing on the causality of languages and their context dependence (requirement):

$$P(X) = \prod_{i=1}^n P(x_i | x_1, \dots, x_{i-1})$$

Ngram LM based on the *Markov Assumption* (generated token counts as well): $P(X) = \prod_{i=1}^n P(x_i | x_{i-N+1:i-1}) \rightarrow$ e.g. Bigram $P(X) = \prod_{i=1}^n P(x_i | x_{i-1})$.

This prediction is based on the surroundings of the generated tokens by extracting a *probability distribution* given the corpus. The vocabulary V consists of *subword* tokens (e.g. Byte-Pair Encoding), so $|V|$ is fixed and out-of-vocabulary words are split into known pieces.

3.1.1 Count-Based LM

Count how many times a target word appears in different contexts ($C(\text{target} | \text{context}_i)$). Goal is to find some $C(\text{target} | \text{context}_j) \gg C(\text{target} | \text{context}_i)$: *Maximum Likelihood Estimation (MLE)*:

$$P_{MLE}(\text{target} | \text{context}) = \frac{C(\text{context}, \text{target})}{\sum_{x \in V} C(\text{context}, x)}$$

This can be applied to ngram LM:

$$P_{MLE}(x_i | x_{i-N+1:i-1}) = \frac{C(x_{i-N+1}, \dots, x_i)}{C(x_{i-N+1}, \dots, x_{i-1})} \quad \text{Bigram: } P_{MLE}(x_i | x_{i-1}) = \frac{C(x_{i-1}, x_i)}{\sum_{x \in V} C(x_{i-1}, x)} = \frac{C(x_{i-1}, x_i)}{C(x_{i-1})}$$

Since we only approximate the true probability all models are *potentially wrong*, and when N gets large, the counts become very sparse. thus we logarithmize the probabilities to avoid underflow: $P_1 \times P_2 \times P_3 = \exp(\log P_1 + \log P_2 + \log P_3)$

Issue: Many Ngrams will never occur in the corpus, thus $P(X) = 0$. We can apply

Smoothing. Reserve probability mass for unseen ngrams by treating rare events as if seen more often (*add artificial counts*), while preserving $\sum_i p_i = 1$:

- *Laplace (add-1)*: $P(x_i | x_{i-1}) = \frac{C(x_{i-1}, x_i) + 1}{C(x_{i-1}) + |V|}$ — add 1 to every count; over-smooths when $|V|$ is large.
- *Add-k*: $P(x_i | x_{i-1}) = \frac{C(x_{i-1}, x_i) + k}{C(x_{i-1}) + k|V|}$ — tunable $k < 1$ softens the over-smoothing.
- *Kneser-Ney*: discount each seen count by a fixed δ and back off to a lower order,

$$P_{KN}(x_i | x_{i-1}) = \frac{\max(C(x_{i-1}, x_i) - \delta, 0)}{\sum_{x \in V} C(x_{i-1}, x)} + \lambda_{x_{i-1}} P_{KN}(x_i)$$

, where the lower-order term is a *continuation probability* (how many distinct contexts x_i follows), not the raw unigram.

Interpolation & backoff. The zero-count problem ultimately stems from contexts being too long. Rather than trusting a single ngram order, mix all orders:

$$\hat{P}(x_i | x_{i-2}, x_{i-1}) = \lambda_1 P(x_i) + \lambda_2 P(x_i | x_{i-1}) + \lambda_3 P(x_i | x_{i-2}, x_{i-1}), \quad \sum_* \lambda_* = 1,$$

with the λ s estimated on held-out data via **Expectation Maximization**. In short: *smoothing* redistributes mass *within* a fixed context, *interpolation* mixes *across* context lengths — Kneser-Ney does both.

Cons.

- Bad probability estimation: very large corpora required
- Curse of dimensionality: memory intense to store counts
- Limited Context length: Markov assumption
- Smoothing dependency: we always shift some

3.1.2 Auto-regressive LM

Replace *counting* with high-dimensional *word embeddings* to model the same chain-rule factorization

$$P(X) = \prod_{i=1}^n P(x_i | x_{<i}).$$

Instead of MLE counts, we maximize the likelihood of proper language via the **cross-entropy loss**:

$$\mathcal{L}_\theta = -\frac{1}{N} \sum_{i=1}^n y_i(x_i) \log P(x_i | x_{<i})$$

where $y_i(x_i)$ is the true (*one-hot*) distribution $\rightarrow$ only the true token's term survives, so we simply push up $\log P$ of the correct next token.

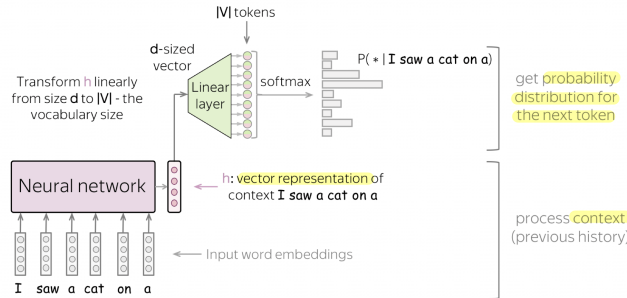


Figure 2: Auto-Regressive Next-Token-Prediction

Forward pass (per token x_i).

1. **Embed:** $(x_1, \dots, x_n) \mapsto (e_{x_1}, \dots, e_{x_n})^T$ via the embedding layer
2. **Contextualize:** hidden repr. of the prefix $h_{<i} = f(e_{x_1}, \dots, e_{x_{i-1}})$, with f the network (e.g. Transformer)
3. **Predict (softmax):** score each vocab token by its similarity to $h_{<i}$

$$P(x_i | x_{<i}) = \frac{\exp(e_{x_i}^T h_{<i})}{\sum_{x \in V} \exp(e_x^T h_{<i})}$$

4. **Loss & update:** backpropagate $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_\theta$

Self-supervision: the labels are the next tokens themselves — *language is its own label* $\rightarrow$ massive pre-training without manual annotation (the key idea enabling LLMs).

Decoding strategies. Generation samples token-by-token until $\langle \text{eos} \rangle$. How we pick from $P(x_i|x_{<i})$:

- **Greedy:** $x_i = \arg \max_x P(x|x_{<i})$ — deterministic, but repetitive/bland.
- **Top- k :** keep the k highest, re-normalize over them, sample: $P'(x_i|x_{<i}) = \frac{P(x_i|x_{<i})}{\sum_{j \in \text{top-}k} P(x_j|x_{<i})}$. $k=1$ = greedy; larger k = more diverse.
- **Top- p / nucleus:** smallest set $\mathcal{N}$ with cumulative mass $\sum_{x \in \mathcal{N}} P(x|x_{<i}) > p$, then sample $\rightarrow$ *dynamic* candidate count ($p=0.9$ common).
- **Temperature τ :** rescale logits before softmax, $\frac{\exp(e_{x_i}^T h_{<i}/\tau)}{\sum_{x \in V} \exp(e_x^T h_{<i}/\tau)}$. $\tau < 1$ sharpens (more deterministic), $\tau > 1$ flattens (more diverse), $\tau=1$ unchanged.

3.1.3 LM Performance Evaluation

Goal: measure how well an LM generates proper language (higher probability). Log-likelihood $\sum_{i=1}^n \log P(x_i|x_{<i})$ is possible but it grows with length and is not interpretable.

Perplexity. The length-normalized, exponentiated negative log-likelihood:

$$\text{PPL}(X_\theta) = 2^{-\frac{1}{n} \sum_{i=1}^n \log_2 P(x_i|x_{<i})} = \exp\left(-\frac{1}{n} \sum_{i=1}^n \log P(x_i|x_{<i})\right)$$

Equivalently via entropy: $\text{PPL}(X_\theta) = 2^{H(X_\theta)}$ with $H(X_\theta) = -\frac{1}{n} \sum_{i=1}^n \log_2 P(x_i|x_{<i})$, i.e. perplexity is just the (exponentiated) cross-entropy loss, so the eval metric *is* the training objective.

- **Intuition:** perplexity is the average *number of choices* the LM considers per step: $\text{PPL} = 8 \approx$ choosing uniformly among 8 tokens. Lower = less "perplexed" = better.
- **Base:** $\log_2$ (bits: "how many bits to encode each token") for perplexity; training likelihoods often use $\ln$. The base only rescales and does not change the ranking.

Bounds $\text{PPL}(X_\theta) \in [1, |V|]$:

- **Lower = 1:** $H = 0$, no uncertainty; a perfect, fully confident LM (not perplexed at all).
- **Upper = $|V|$:** worst case is uniform random guessing $P = \frac{1}{|V|} \Rightarrow \text{PPL} = 2^{-\frac{1}{n} \cdot n \cdot \log_2 \frac{1}{|V|}} = |V|$. Picking randomly from the (tokenizer) vocabulary.

3.2 LM Architectures

Chronology: Encoder-Decoder (Seq2Seq, 2014) $\rightarrow$ Encoder-Only (BERT, 2018) $\rightarrow$ Decoder-Only (GPT, 2018).

From SMT to NMT. Statistical MT decomposes via Bayes

$$\theta_{SMT} : \arg \max_y P(y|x) \approx \arg \max_y \underbrace{P(x|y)}_{\text{translation}} \underbrace{P(y)}_{\text{LM}},$$

needing a reverse model + LM $\rightarrow$ heavy hand-crafted rules (alignment, dictionaries, morphology). **Neural MT** models $P(y|x)$ *directly*, no decomposition.

Encoder-Decoder. Seq2Seq (I/O view) = Encoder-Decoder (model view); encoders evolved CNN $\rightarrow$ RNN $\rightarrow$ Transformer (also T5, BART). Transformer backbone:

- **Positional Encoding** (inject order): $PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$, $PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$
- **Self-Attention:** $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$
- **Multi-Head:** $\text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$, with $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Encoder-Only (BERT). Motivation: want *representations* not generation, and autoregressive Transformers are *unidirectional* (late context can't inform early). **Bidirectional Encoder Repr.** from Transformers, trained with **Masked LM**: 15% of tokens selected (very inefficient!), of which 80% $\rightarrow$ [MASK], 10% $\rightarrow$ random token, 10% unchanged; predict only the masked tokens (all at once).

Decoder-Only (GPT). Causal, autoregressive (CLM) $\rightarrow$ strongest zero-shot generalization after self-supervised pre-training; the SOTA choice. Decoder-only > encoder-decoder:

- Enc-Dec: task-tailored (reduced generality), slow inference, task-specific objectives (costly).
- Dec-Only: unified objective (CLM loss), efficient (only the decoder), scales easily (scaling law), versatile via **prompting** (any classification $\rightarrow$ generation task).

T5 (enc-dec) vs GPT-3 (dec-only): explicit task-specific vs implicit free-form prompts; no vs yes few-shot; fine-tuning on curated data vs pre-training on arbitrary text; hard vs easy to scale.

3.3 Scaling Law

Model perspective: emergent abilities of LLMs. New, sophisticated capabilities that are *not explicitly trained*, appearing only past a scale threshold — scale-dependent, emergent, non-programmed, data-driven. Scaling is *linear* in the encoder-decoder era but *non-linear* (sharp jumps) in the decoder-only era.

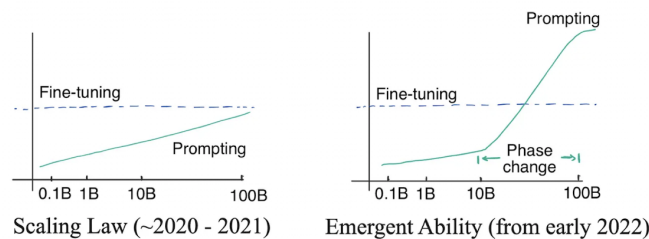


Figure 3: Scaling Law: Left in the Encoder-Decoder Period (linear); Right in the Decoder-Only Period (non-linear)

Data perspective: compute-optimal training. Under a fixed compute budget, how to trade model size N vs. training tokens D ? A power-law links N , D , and performance:

- **OpenAI (Kaplan):** model size $\approx 3\times$ more important than tokens $\rightarrow$ scale the model.
- **Chinchilla:** optimal token-to-parameter ratio $\approx 20:1 \rightarrow$ scale N and D *equally*.
- **Llama-3:** pushes the ratio to $\approx 215:1$ (deliberate *overtraining*).

Over- vs undertraining: overtraining costs more compute but yields more capable (often smaller) models; today's LLMs are in fact heavily *undertrained* (100–1000 $\times$ from convergence).

Inference scaling (new trend). Spend more compute at *inference* time (not only training) for better performance.

Open directions: optimal token-parameter ratio, data quality/efficiency, inference scaling, novel architectures (RWKV, RetNet, Mamba = selective state spaces).

3.4 LLM Paradigms

Two stages of the LLM life cycle: *learning* paradigms change the parameters (training), *reasoning* paradigms leverage the frozen parameters to solve tasks (inference).

3.4.1 Learning Paradigms

Pre-training + Fine-tuning. The classic two-stage *transfer learning* recipe:

- **Pre-training:** massive *unlabeled* data, self-supervised (auto-regressive CLM); the model picks up syntax, semantics, world knowledge.
- **Full-parameter fine-tuning:** small *labeled* dataset, supervised; adapts the model to a specific task.

Worked well on *intermediate* tasks (POS tagging, NER, coreference, parsing), but two problems: (1) modeling intermediate tasks separately is inefficient and does not solve *end* tasks end-to-end; (2) it is *rigid*, we must query the model the way it was trained $\rightarrow$ we want LLMs to follow human instructions, with minimal train-inference mismatch and updatable knowledge (no catastrophic forgetting).

Teacher forcing (aside): during training condition on the gold tokens, not the model’s own predictions $\rightarrow$ fast convergence, but it is the root of the train-inference mismatch.

Instruction Tuning. Full-parameter fine-tuning on {Instruction}-{Input}-{Response} triplets with CLM. Any task is reformulated into this format, so adapting to human-like instructions mitigates the train-inference mismatch.

$$\mathcal{L}_\theta = -\frac{1}{R} \sum_{j=1}^R \log P(y_j | y_{<j}, x; \theta), \quad x = \text{Instruction} + \text{Input}, \quad y = \text{Response}$$

- **Completion-only** (loss on the response only): better instruction-*following*.
- **Full-prompt** (loss on the whole sequence): better output-*structuring*.
- Many instruction datasets are 100% *synthetic* (LLM-generated).

Mixture of Experts (aside): replace the FFNN with an MoE layer (one router + many experts, only a few activated per token) $\rightarrow$ huge parameter count but *sparse* activation, so faster inference.

Parameter-Efficient Fine-Tuning (PEFT). Instruction tuning still updates *all* parameters. PEFT freezes the pre-trained model and trains only a small set of extra parameters.

- **Prefix Tuning:** prepend a trainable prefix matrix $P_\theta \in \mathbb{R}^{|P_{idx}| \times \dim(h_i)}$ to the activations at *every* layer ($\approx 1\%$ of params):

$$h_i = \begin{cases} P_\theta[i, :], & i \in P_{idx} \\ \text{LM}_\phi(z_i, h_{<i}), & \text{otherwise} \end{cases}, \quad z = [\text{PREFIX}; x; y]$$

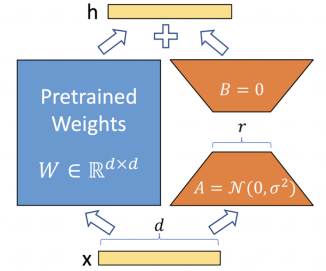
PrefixLM = CLM + prefix; the prefix is not inherently causal (+BERT stays MLM, +GPT stays CLM).

- **Prompt Tuning:** train only the embeddings of *soft prompt* tokens (numerical, not human-readable) prepended at the input level. *Hard prompts* are human-readable natural-language tokens.
- **LoRA** (Low-Rank Adaptation): task adaptation has low *intrinsic rank*, so the update ΔW is itself low-rank. Freeze W_0 and learn only the factorized adapter $\Delta W = BA$:

$$h = W_0 x + \Delta W x = W_0 x + B A x, \quad W_0 \in \mathbb{R}^{d \times k}, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k)$$

Trainable params drop from $d \times k$ to $r(d + k)$. At inference ΔW can be *merged* back into W_0 (use $W_0 + BA$) $\rightarrow$ no added latency.

- **QLoRA:** LoRA on a quantized base model (further memory savings).



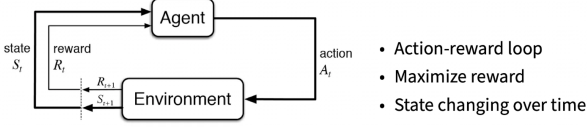
	Instruction	Prefix	Prompt
Prompt type	Hard	Soft	Soft
Training aim	Instruction-following	Tuning efficiency	Tuning efficiency
Interpretability	High	Low	Medium
Prepend location	Input layer	Activations (every layer)	Input

Table 1: Instruction vs. Prefix vs. Prompt tuning.

Prompt engineering: search for the best static *hard* prompts (readable + performant); designing the prepended soft vectors is *soft* prompt engineering.

Alignment (RLHF). Instruction tuning and PEFT still rely on teacher forcing (likelihood $\neq$ human preference) and cannot encode what humans actually *want*. **RLHF** fine-tunes without teacher forcing by optimizing a human-preference reward, targeting ethics (no toxic output), faithfulness, and fewer hallucinations.

Generation as RL: a sequential decision process with the reward granted at the sequence level.



RL	RL for LLMs
Agent	the LLM
Environment	input prompt
State	tokens generated so far $x_{<j}$
Action	select the next token
Reward	preference feedback signal

(1) Reward modeling (Bradley-Terry). Humans label preference pairs (x, y_w, y_l) (winner $\succ$ loser). The preference probability is a sigmoid of the *reward gap*:

$$P(y_w \succ y_l \mid x) = \sigma(r^*(x, y_w) - r^*(x, y_l))$$

The true r^* is intractable, so fit a proxy reward model r_ϕ by maximizing this likelihood:

$$\mathcal{L}_R = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

Then optimize the policy to maximize reward, with a *KL penalty* keeping it near the SFT reference (a trust region that prevents reward hacking / degeneration):

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta} [r_\phi(x, y)] - \beta D_{KL} [\pi_\theta(y \mid x) \parallel \pi_{ref}(y \mid x)]$$

(2) PPO optimizes that objective. *Intuition*: raise the probability of high-advantage actions, but *clip* the policy ratio so each step stays in a trust region:

$$\mathcal{L}_\theta^{clip} = \mathbb{E}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)], \quad r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{old}(a_t \mid s_t)}$$

with advantage $\hat{A}_t$ and an added entropy bonus (exploration). *Cost*: 3 models held in RAM at once $\rightarrow$ heavy and unstable:

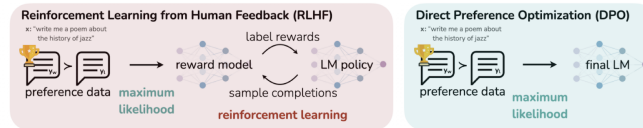
- **Policy** π_θ : initialized from SFT, the only model PPO trains.
- **Reference** π_{ref} : a frozen SFT copy, anchors the KL penalty.
- **Reward** r_ϕ : trained on human preferences, frozen, scores the generated outputs.

(3) Simplifications progressively remove components:

- **DPO** (Direct Preference Optimization): the optimal RLHF policy implies the reward is $r(x, y) = \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{ref}(y \mid x)} + \text{const}$, so the *policy is its own implicit reward*. Substituting into Bradley-Terry collapses RLHF into one classification loss on pairs:

$$\mathcal{L}_{DPO} = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{ref}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{ref}(y_l \mid x)} \right)$$

Intuition: push the winner's log-prob up and the loser's down, each measured *relative to the frozen reference*. *No reward model, no RL loop*; load two models, train one.



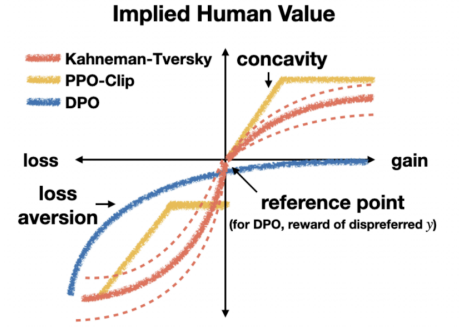
- **SimPO**: DPO still needs π_{ref} to form its reward. SimPO drops it, using the policy's own *length-normalized* average log-prob as the implicit reward $r(x, y) = \frac{\beta}{|y|} \log \pi_\theta(y \mid x)$. *Even lighter* (one model only); dividing by $|y|$ removes length bias and matches the reward to what decoding actually optimizes.
- ***PO zoo** (IPO, CPO, ORPO, R-DPO, GRPO): all just different ways to push preferred-output probability up. **GRPO** (Group Relative Policy Optimization) is PPO that samples a *group* of outputs per prompt and scores each by its reward *relative to the group average*, dropping the separate value/critic model; this group-relative baseline underlies modern reasoning RL.

(4) **KTO** (Kahneman-Tversky Optimization) changes the *feedback form*: binary good/bad labels instead of pairs, scored by a prospect-theory utility (humans are *loss-averse*: value concave for gains, convex for losses). *Intuition*: aligns to human utility, and binary signals are far cheaper to collect than pairwise preferences.

$$\mathcal{L}_{KTO} = \mathbb{E}_{(x,y) \sim \mathcal{D}} [1 - \hat{h}(x, y; \beta)]$$

$$\hat{h} = \begin{cases} \sigma(\beta \log \frac{\pi_{\theta}(y|x)}{\pi_{ref}(y|x)} - \beta D_{KL}), & y \text{ desirable} \\ \sigma(\beta D_{KL} - \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{ref}(y|x)}), & y \text{ undesirable} \end{cases}$$

with $D_{KL} = D_{KL}(\pi_{\theta} \parallel \pi_{ref})$: desirable outputs maximize $\hat{h}$, undesirable ones minimize it.



	Feedback	Core idea	Gains/losses
DPO	Preference pairs	Relative preference via $\log(\pi_{\theta}/\pi_{ref})$	Linear, symmetric
SimPO	Preference pairs	DPO without the reference model	Like DPO, cheaper
KTO	Binary (good/bad)	Prospect-theory human utility	Nonlinear, loss-averse

Table 2: Preference-optimization methods.

3.4.2 Reasoning Paradigms

How a *frozen* model uses its knowledge at inference time (no parameter change), as opposed to learning paradigms that change the weights.

Chain-of-Thought (CoT). Prompt the model to generate explicit intermediate reasoning steps instead of answering directly. *Intuition*: mimics human problem solving (decompose $\rightarrow$ reason step-by-step $\rightarrow$ combine), making the output transparent and letting external steps (e.g. retrieval) plug into the chain.

- A *hard-prompt* technique (e.g. “Let’s think step by step”), not training.
- **Prerequisites**: only works on capable models, i.e. *emergent* (scaled-up, good zero/few-shot) and *instruction-tuned* (can follow the prompt).
- Evaluated on reasoning benchmarks (math: GSM8K, MATH; logic: CLUTRR; code: CRUXEval).
- *Trend*: large-scale RL alone (no SFT) can elicit strong reasoning $\rightarrow$ ability to generalize $\approx$ ability to reason.

In-context Learning (Few-shot). Put a few worked examples directly in the prompt; the model infers the pattern *at inference*, with no weight updates. Combines naturally with CoT (few-shot reasoning exemplars).

LLM Agents. An (instruction-tuned) LLM that interacts with an environment, makes decisions, and takes actions to solve tasks. *Intuition*: offload what the LLM is bad at (exact computation, fresh facts) to external tools and let it orchestrate. Components:

- **Agent**: a general-purpose LLM, set up via a role prompt.
- **Tools**: APIs, code interpreter, calculator, databases, an IR/MT system, even another agent.
- **Memory**: *short-term* (in-context, recent steps) and *long-term* (external vector DB for retrieval).
- **Planning**: decompose into subtasks via CoT or Tree-of-Thought (ToT); single-path vs multi-path.

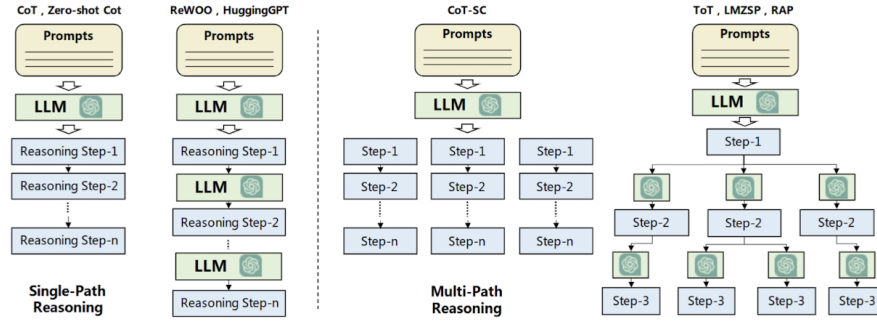


Figure 4: Single-Path Planning vs. Mutli-Path Planning

Evaluation: human scoring, task metrics, Turing test, agent benchmarks (ToolBench, AgentBench). The long-term-memory-as-retrieval component leads directly to **Agentic RAG**.

4 RAG Engineering

Engineering a deployable RAG system end to end, in four stages: **build** it (backend: data, retrieval, APIs), **expose** it (frontend: UX), **verify** it (testing), and **extend** it (frameworks and application patterns).

4.1 Backend Engineering

The engine of the system: *database management* (how data is stored, retrieved, and ranked), the *system/API design* that serves it, and *deployment*.

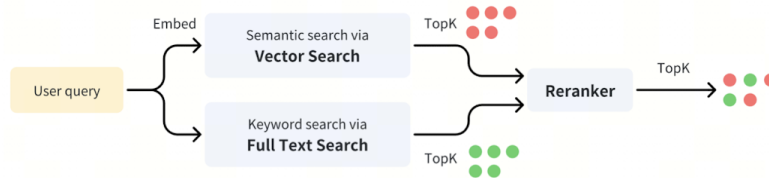
4.1.1 Database Management

Database: a system for persistently storing data so it can be queried, updated, and controlled efficiently. Two requirements: *structure* (easy to manipulate/extend) and *efficiency* (fast, precise fetch).

SQL. Standard language for relational DBs. Core verbs: **SELECT** (which columns), **FROM** (which table), **WHERE** (which rows), **ORDER BY** (sort), **LIMIT** (how many). Document stores (e.g. MongoDB = collections of JSON objects) expose Python interfaces (**pymongo**) for backend integration.

Search strategy.

- **Keyword** (BM25): precise and fast, but *low recall* (rigid, exact terms only).
- **Semantic** (embeddings): captures meaning → high recall; the default RAG backbone.
- **Hybrid:** BM25 *pre-fetch* (fast over millions of docs) + semantic *re-rank* (precise), or run in parallel → high precision *and* rec¹¹



Chunking. The granularity at which documents are stored for retrieval, the *most important step* of building the DB. *Central tradeoff:* chunks too small lose context, too large fuse unrelated content and add noise. Finer granularity also means more chunks and more compute.

Strategy	Main idea	Pros / Cons	When to apply
Fixed-size	Chunks of fixed token/char length	+ simple, fast, scalable – breaks semantic units, boundary errors	Baseline systems, small datasets, low-latency / cost-constrained
Sliding-window	Fixed chunks overlapping by a stride	+ keeps cross-boundary context, more coherent – redundant (many overlapping chunks)	Sequential / conversational text where boundary context matters
Hierarchical	Chunks across levels (section → paragraph → sentence)	+ strong grounding, explainable – complex, more storage, hierarchy can blow up	Long structured documents, high faithfulness, explainable RAG
Semantic	Split where semantic change exceeds a threshold	+ topic-aware, adaptive sizes – embedding cost, threshold-sensitive	Heterogeneous documents, topic shifts, high-quality retrieval

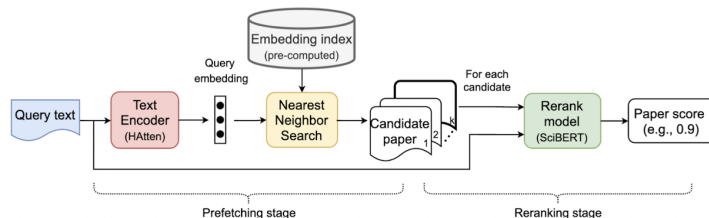
Table 3: Chunking strategies.

Semantic chunking concretely: embed and normalize sentences, compute adjacent distances $d_i = 1 - \cos(v_i, v_{i+1})$, set threshold T as the p -th percentile of $\{d_i\}$, then start a new chunk whenever $d_i \geq T$ (subject to a min/max chunk size, with 10–20% overlap between chunks).

Encoding. Encoders have limited input length, another reason to chunk. Even with long-context encoders, chunk to (i) retrieve only relevant content (less noise) and (ii) cut token/traffic cost.

Indexing. Organize stored vectors so queries map to objects efficiently, e.g. approximate nearest-neighbor search.

Re-ranking (two-stage retrieval). Pre-fetch top- K fast with the ANN index, then re-score that shortlist with a stronger model. *Why not retrieve directly with the strong model:* too slow at scale, vector indexes optimize speed not fine-grained relevance, and re-ranking corrects ANN approximation + embedding noise. Techniques: **cross-encoder** (joint query-doc encoding, highest accuracy), **LLM-as-a-Judge**, **bi-encoder** re-scoring (cheap), **heuristic** (metadata, recency, authority, length).



Query Optimization. User queries are short/ambiguous, and vector search depends entirely on the query embedding (garbage in $\rightarrow$ garbage out). Refine before encoding: *expansion* (add concepts/synonyms), *reformulation* (clarify intent/structure), *metadata-aware* (inject filters/constraints).

Prompt Design. A good prompt has a clear task specification, structured input (helps attention), and a defined output format.

4.1.2 System Design & APIs

RAG design principles.

- **Modular architecture:** independent, plug-and-play modules with stand-alone compute (APIs) and a unified data-exchange format. *No API does more than one task.*
- **High-quality retrieval:** combine semantic (embedding) and keyword (index) search, i.e. hybrid.
- **Effective query formulation:** dynamic reformulation from user feedback; query normalization (misspellings, etc.).
- **Scalability & efficiency:** scale up/down to fit demand; minimize end-to-end latency.

APIs & REST. An **API** is a gateway letting programs communicate; the *protocol* defines the rules for client (frontend) $\leftrightarrow$ server (backend) exchange. **REST** (REpresentational State Transfer) is an architectural style over HTTP/HTTPS (always HTTPS, i.e. HTTP + TLS encryption). Methods: **GET** (retrieve; 200/404/400), **POST** (create), **PUT** (replace), **PATCH** (partial update), **DELETE** (remove).

```
from flask import Flask
app = Flask(__name__)           # __name__ lets Flask locate files/config

@app.route("/")                 # register an endpoint
def hello_world():
    return "<p>Hello, World!</p>" # auto-wrapped into an HTTP response
```

Run with `python api-server.py` (serves on port 5000); `--host=0.0.0.0` to listen on all public IPs; never `--debug` in production (allows browser-side code execution).

API routing. Test backend communication first (a **GET** / status check + a **POST** /echo) before building complex routes. Name endpoints by *resource* + *action*, sharp and precise: good `/query`, `/retrieve`, `/rank`, `/generate`; bad `/qwen-7B`, `/my-ranker`, `/rag-project/backend/search/documents`. *Why it matters:* keep modules one-task and let APIs communicate only when necessary (each call adds latency); follow minimalism (no redundant components) and never build on a fragile backend.

GraphQL. REST has two pain points at scale: *many endpoints* to manage, and *over-fetching* (a fixed endpoint returns more fields than needed). **GraphQL** is a query language where the client requests *exactly* the data it needs from a *single* endpoint:

1. server defines a typed data *schema*;
2. client writes a query naming the desired fields;
3. server returns precisely those fields, nothing more.

Tradeoff: eliminates over-fetching and endpoint sprawl, but shifts complexity to the server (schema + resolvers) and makes caching harder than REST’s per-endpoint model.

4.1.3 Deployment

Local vs. cloud.

- **Local** (dev/testing, internal use): easy to debug/maintain and safer (data stays in-house); but limited compute for large models and no public access (few users).
- **Cloud** (commercial/institutional): scalable compute and public reach; but costs money and is subject to **GDPR** / data-protection rules (apply in Switzerland too, especially for sensitive human data).

Docker. *Problem:* re-configuring inter-dependent, version-specific packages (plus CUDA driver mismatches) on every machine, the “it-works-on-my-machine” trap. **Docker** gives a consistent, portable runtime: the same behaviour across dev/test/prod, on any platform. Three concepts:

- **Docker Engine:** runtime that runs containers on any system.
- **Docker Image:** read-only template (application code + environment).
- **Docker Container:** a running instance of an image, self-sufficient (code, runtime, libraries, config).

Unlike a VM, containers *share the host OS kernel* (no guest OS) → lighter, faster, denser (*Pack-Build-Ship-Run*).

	Docker container	Virtual machine
Isolation	low	high
Launch time	seconds	minutes
Image size	several MB	hundreds of MB to GB
Portability	platform-agnostic	platform-dependent
Density (per machine)	100–1000	10–100

Table 4: Docker containers vs. virtual machines.

4.2 Frontend Engineering

Turns the engine into a usable product: design principles, the React.js building blocks, and the HCI patterns RAG is actually deployed as.

4.2.1 Design Principles

Users judge the *experience*, not the backbone. The frontend’s job is **UX**, but a seamless UX needs frontend and backend working together (failure in either breaks it).

- **UI** (“how it looks”): layout, colors, typography, components, spacing, micro-interactions (hover, focus, animation).
- **UX** (“how it feels”): task flow, latency perception/feedback, error handling/recovery, trust and clarity.

Principles: know the building blocks (more than vibe coding); *responsive* design (fit every screen); *unified* styling; UX optimization (user research → prototyping → consistency → feedback/iteration); accessibility. *RAG-specific:* surface loading/progress feedback (slow answers), warn on low-confidence output (hallucination), show citations (verifiability), and give recovery hints when retrieval fails.

Core components. A frontend codebase is three layers: **HTML** (skeleton, what exists), **CSS** (body, how it looks), **JavaScript** (CNS, how it acts). Frameworks like React.js do not replace them, they *compose* them into reusable components.

4.2.2 React.JS

Core idea: UI is a *function of state*; a state change triggers React to re-render automatically. React tracks changes via a *virtual DOM* (HTML parsed into a tree, each component a node) and patches *only the necessary* changes, not the whole page. This is *declarative* (describe the UI per state) vs Vanilla JS’s *imperative* (manually select and mutate elements).

State = data that changes over time (user interactions, network responses, streaming updates). *Not state*: constants, derived values, temporary variables, DOM elements.

useState. Adds a state variable: `const [state, setState] = useState(initial)`. `state` is the value for the current render, `setState` *requests* an update (never reassign manually). Gotcha: calls are *batched* against the current snapshot, so

```
setAge(age + 1); setAge(age + 1);           // both read age=28 -> 29 (not 30)
setAge(a => a + 1); setAge(a => a + 1);      // updater queue -> 28->29->30
```

Use the *updater function* form when the next state depends on the previous one.

useContext. Passing state down through components that don’t need it (*prop drilling*) is painful because component structure $\neq$ data flow. Context shares a bundle: define it (`createContext`), a `Provider` wraps the top, and any descendant consumes only what it needs.

```
const { studentName } = useContext(UniversityContext);
```

Caveat: any change in the bundle re-renders *all* consumers, so don’t put frequently-changing values in one giant context.

useEffect. Synchronizes a component with an *external system*: runs after render, re-runs when its dependencies change, with an optional cleanup. The natural place to talk to the RAG backend, e.g. re-retrieve whenever the query changes:

```
useEffect(() => {
  if (!query) return;
  fetch(`/retrieve?q=${query}`).then(r => r.json()).then(setDocuments);
}, [query]);           // [] = run once; [query] = re-run on change
```

4.2.3 Human-Computer Interaction

RAG is an economical route to grounded knowledge: no fine-tuning if the base model is good, plug-and-play with new models, customizable and scalable. Two common frontends:

RAG as Chatbot. Conversational interface where answers are generated from retrieved knowledge (NL input, turn-based, grounded in cited sources). Chatbots already handle open-ended and follow-up interaction; RAG adds factual grounding, up-to-date info (web search), domain knowledge, and reduced hallucination. The UI hides the complexity (chat history, current answer, optional sources) while each query secretly triggers a full RAG cycle (latency and uncertainty are real). *Separate configuration* (model choice, decoding, prompt templates) *from execution*.

RAG as Data Visualizer. A data-driven canvas where answers are structured, inspectable visualizations (tables, charts, dashboards) instead of text, hence much more frontend-heavy. The same RAG backend can power both, but the frontend architecture differs fundamentally.

	RAG Chatbot	RAG Data Visualizer
Primary output	Natural-language text	Tables, charts, structured views
Interaction	Conversational, turn-based	Exploratory, filter/control-driven
Result persistence	Low (ephemeral)	High (inspectable)
Trust mechanism	Citations alongside text	Structure, aggregation, provenance
Cognitive load	Higher (recall context)	Lower (info externalized)

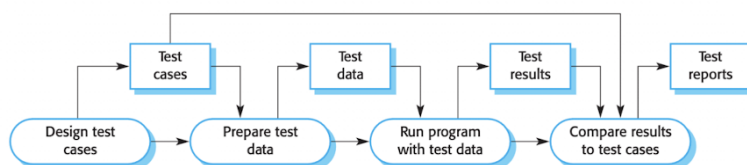
Table 5: RAG chatbot vs. data visualizer.

Design for the full lifecycle. Cover *idle* → *retrieving* → *ranking* → *generating* → *done/error*, not just the final answer: show progress feedback, always support stop/retry/regenerate, and reduce cognitive load with structure (citations + evidence cards; tables + filters). A good RAG frontend helps users *verify, steer, and recover*, not just read.

Vibe Coding. Coding agents let anyone prototype, but *vibe first* (sketch, feel the flow), then *stabilize after* (states, transitions, latencies). Use agents for scaffolding, mock data, UI exploration, refactoring, and docs, but you must still own the data flow, why/when each hook runs, failure-and-recovery, and where trust/evidence come from. *Code agents write code; you are the architect.*

4.3 System Testing

With the system built and exposed, verify it. Every test answers one of two questions: *validation* (does the software do what we want?) and *defect* (does it do things correctly?). Commercial systems pass three stages: **development**, **release**, and **user** testing.

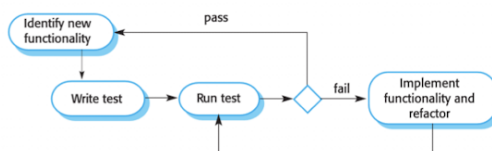


4.3.1 Development Testing

Done by developers during development to find bugs.

Test-Driven Development (TDD). Build incrementally with a test per increment, and do not start the next increment until the current one passes. Cycle: map an increment to a code chunk (a frontend component, a backend endpoint) → write and run its test → pass: submit a PR and merge; fail: debug and re-implement → branch for the next increment. *Why it matters:*

- code→function mapping enforces modularization (the foundation of debugging);
- *regression testing*: re-run old tests to catch bugs introduced by new increments (cheap);
- more organized, structured documentation and test logs.



Hierarchical levels.

- **Unit testing:** individual units (classes, functions), focus on *functionality*. Design cases by *input partitioning* (group inputs treated alike) and test on *partition boundaries* (empty input, exact-size, over-size).
- **Component testing:** a component of related units, focus on the *interface* (parameter passing). Verify that information survives the journey between components (correct types, required fields present, parameters unchanged), not the internal functionality or output quality.
- **System testing:** integrate components into a version and check compatibility, interaction, and data flow. Collaborative and run by an *independent* team (developers are biased to their own happy path and ignore edge cases).

On-the-fly optimization: don't wait for system-test results to optimize (too late). Start from the best available solution (SOTA models/frameworks) and trade off performance vs. latency, driven by cost not perfectionism.

A minimal unit test (pytest-style), exercising a boundary partition:

```

def test_chunk_long_text():
    chunker = DocumentChunker(max_tokens=3)
    chunks = chunker.chunk("retrieval augmented generation systems require testing")
    assert all(len(c.split()) <= 3 for c in chunks) # size constraint respected

```

4.3.2 Release Testing

A separate team checks a complete version before production. System testing asks “does it work as specified?”; release testing asks “is it good enough for the market?”.

Aspect	System testing	Release testing
Goal	Correctness of integrated system	Readiness for release
Perspective	Developer / engineer	User, customer, organization
View	Product with known internals	Black box with a spec
Environment	Controlled, internal	Production-like / staging
Focus	Functional behavior	Fitness for use
Failure tolerance	Low (must fix)	Some known issues accepted
Outcome	Pass/fail vs. spec	Go / no-go decision

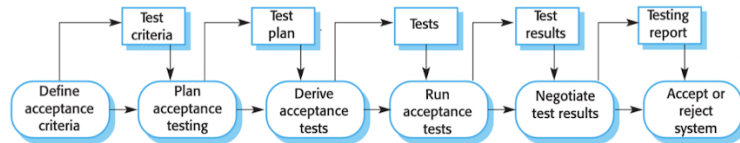
Table 6: System testing vs. release testing.

Three categories:

- **Requirements-based:** verify functional/non-functional requirements that matter to users (answer with citations; abstain when no evidence; safety disclaimer for legal/medical; response under a threshold).
- **Scenario:** end-to-end user interactions, including ambiguity, follow-ups, multi-hop, out-of-DB questions, and imperfect inputs.
- **Performance:** latency (e.g. $P95 < 5s$, $P99 < 8s$), throughput/concurrency, per-query cost.

4.3.3 User Testing

Users test in their own environment; **UX** is the focus (vs. release testing’s functional readiness). Exploratory and observational, so no test cases are prepared in advance.

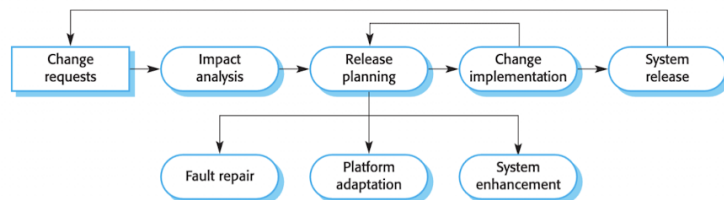


- **Alpha:** a selected user group works closely with the dev team on early releases.
- **Beta:** a release goes to a larger group for UX feedback.
- **Acceptance:** decide whether to deploy in the customer environment (define criteria → plan → run → negotiate → accept/reject).

Versioning `major.minor.patch`: major = breaking changes, minor = non-breaking new components, patch = small fixes.

4.3.4 Software Evolution

Good software has a long lifespan and keeps evolving.



Evolution processes. No standard process exists (maintenance needs and requirements shift), but generally accepted changes spawn a new version, each stage is tested, and new documentation ships with it. **Lehman’s laws:** a system must evolve or become progressively less useful; complexity must be controlled or the system degrades; evolution is feedback-driven. *Evolution* is the long-term trajectory (architectural drift, technical debt, design erosion); *maintenance* is the individual changes.

Software maintenance. Three types:

- **Fault repair** (bugs/vulnerabilities); coding errors $\ll$ design errors in cost.
- **Environmental adaptation** (new platforms, hardware/OS changes).
- **Functionality addition/modification** (business or organizational changes).

Challenges: legacy code, loss of original developers, outdated docs, tight coupling / hidden dependencies. *Tips:* test against multiple runtime versions, pin and document dependencies (`requirements.txt`), add regression tests for fixed bugs. Most effort happens *after* the first release.

4.4 RAG Frameworks & Applications

Beyond a hand-built pipeline, this *extends* the system. A survey chapter: full systems built mostly by recombining earlier primitives (graphs, ViT + contrastive learning, memory, agents).

4.4.1 Frameworks

LightRAG (Graph RAG). Problem: treating documents as independent chunks *flattens* entity relations and loses cross-text context, so the retriever grabs relevant chunks but cannot weave them into coherent multi-hop answers. Idea: represent text *structurally*, not only semantically, as a knowledge graph $G(V, E)$ (nodes = entities, edges = relations; stored as adjacency matrix/list or hashmap). Built by:

- chunk documents, then prompt an LLM to extract *entities* (keys) and *relations* (typed edges with descriptions);
- attach descriptive *values* to nodes (summaries / paraphrases / explanations, not raw text), **grounded:** no value unless supported by the chunk; one key may hold several values from different contexts; deduplicate keys.

Dual-level retrieval: *low-level* keywords $k^{(l)}$ match specific entities, *high-level* keywords $k^{(g)}$ match broad themes/relations. LightRAG retrieves keys and values (not chunks), gathers neighboring nodes, and an LLM integrates them into the answer.

RAGFlow. Open-source framework centered on *deep document understanding*, hybrid retrieval, agentic workflows, and a visual/no-code interface. Pipeline: raw docs $\rightarrow$ ingestion $\rightarrow$ structured knowledge units $\rightarrow$ indexing (vector/BM25/hybrid). The *ingestion service* does multi-format parsing (PDF, Word/PPT, HTML, Markdown), structural segmentation (layout preservation, table/diagram extraction, OCR), and LLM-assisted enrichment (clean noisy text, normalize terminology, summarize long inputs).

4.4.2 Application Patterns

Multimodal RAG. Text-only retrieval is often insufficient (e.g. medical diagnosis); a visual modality disambiguates and supplies details missing from the text query. Needs two ingredients:

- **Image encoder** (Vision Transformer): split the image into patches $\rightarrow$ linear projection + position encoding $\rightarrow$ Transformer (optionally class-prefix tokens; pre-train + fine-tune); ended CNN dominance in vision.
- **Cross-modal matching:** contrastive InfoNCE over (text, image) pairs (the same objective as Week 3 dense retrieval), building a *joint embedding space* that aligns text/image/audio/video.

System quality is bounded by how well cross-modal alignment is preserved; grounding generation in non-textual evidence reduces ambiguity and hallucination.

Conversational RAG. Chatbot + RAG. Conversational queries are under-specified and context-dependent (“the previous answer”, “that paper’s method”), so the system needs persistent state beyond the current turn:

- **Short-term memory:** recent turns (sliding window or running summarization).
- **Long-term memory:** facts/preferences extracted from conversations, indexed like documents and retrieved only when relevant.

A *router* decides whether a memory search is needed at all (saving token traffic and compute).

Agentic RAG. Simple RAG is a fixed pipeline (Query $\rightarrow$ Retrieve $\rightarrow$ Generate); agentic RAG adds *decision-making, iteration, and control*. The agent decides whether retrieval is needed, what/where to retrieve, which tool to call, when to stop, and what to do next, i.e. it treats the task as a *decision* task, not a *lookup* task. Example (book a heavily-constrained flight): extract constraints into structured state $\rightarrow$ retrieve candidates $\rightarrow$ filter $\rightarrow$ verify $\rightarrow$ iterate $\rightarrow$ generate. It fixes simple-RAG failure modes: missing constraints, inability to iterate when constraints conflict, and weak multi-step verification.